



*Advanced AI-based dynamic Integration
of heterogeneous data for semantic
clusters formalisation in human-centric
intelligent enterprise systems*

Ontology-based supervised BERT-like models for automatic triple extraction

Mario Lezoche

Thomas Pallet

Monday 09th February 2026

Table of content

01

**Introduction &
Pipeline overview**

02

**Dataset
creation**

03

Model training

04

**Evaluation
metrics**

05

**Results &
Analysis**

06

Conclusion

Table of content

01

**Introduction &
Pipeline overview**

02

**Dataset
creation**

03

Model training

04

**Evaluation
metrics**

05

**Results &
Analysis**

06

Conclusion

Introduction

Goal : develop and evaluate an ontology-based, supervised BERT-like pipeline (NLP, ...) for automatic triple extraction from text.

Pipeline overview

Goal : develop and evaluate an ontology-based, supervised BERT-like pipeline (NLP, ...) for automatic triple extraction from text.

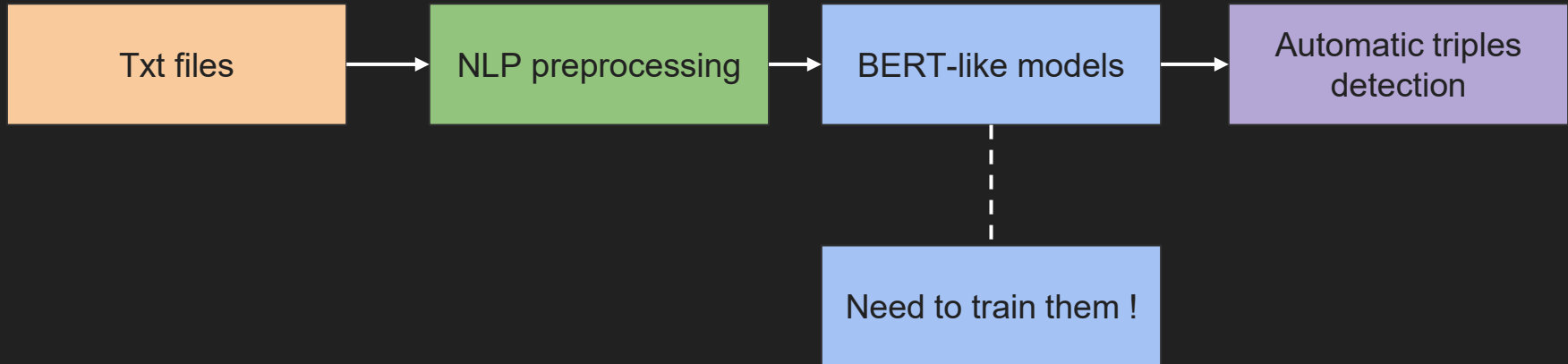


Table of content

01

**Introduction &
Pipeline overview**

02

**Dataset
creation**

03

Model training

04

**Evaluation
metrics**

05

**Results &
Analysis**

06

Conclusion

Dataset creation

- Use of LLMs to create stories, 300 stories (10 lines each)
- Last 50 with more complex knowledge, and always synonym for verbs that are meant to describe a relation (can learn, may learn, is able to learn, ...)
→ multiple templates per relation (would take time to do the per-rule based here)
- Stories are based on the ontology (classes, relations)

Dataset creation

- Entity mention detection
- Pair generation (important to do it bidirectional) because of the way relations works. Entity names E1->E2, only the correct direction is labelled. And added a “None” class for the other direction so that the model doesn’t over classify triples.

- Entity marker injection :

```
{ "text": "[E1]Squirtle[/E1] is typed [E2]Water[/E2].", "label":  
  "http://example.org/pokemon#hasType" }
```

- JSONL exports then for training and validation

Dataset annotation

- Triples : (subject_URI, predicate_URI, object_URI), sentence
- Text are all annotated with JSON
- Feed the model with JSONL format for BERT (80% training and 20% in validation) which gives :
 - 8440 rows | 2188 rows
 - 'http://example.org/pokemon#hasMoveType': 650,
'http://example.org/pokemon#canLearnMove': 650,
'http://example.org/pokemon#ownsPokemon': 448,
'http://example.org/pokemon#hasType': 300,
'http://example.org/pokemon#evolvesTo': 181,
'http://example.org/pokemon#evolvesFrom': 181,
'http://example.org/pokemon#ownedBy': 152,

Table of content

01

**Introduction &
Pipeline overview**

02

**Dataset
creation**

03

Model training

04

**Evaluation
metrics**

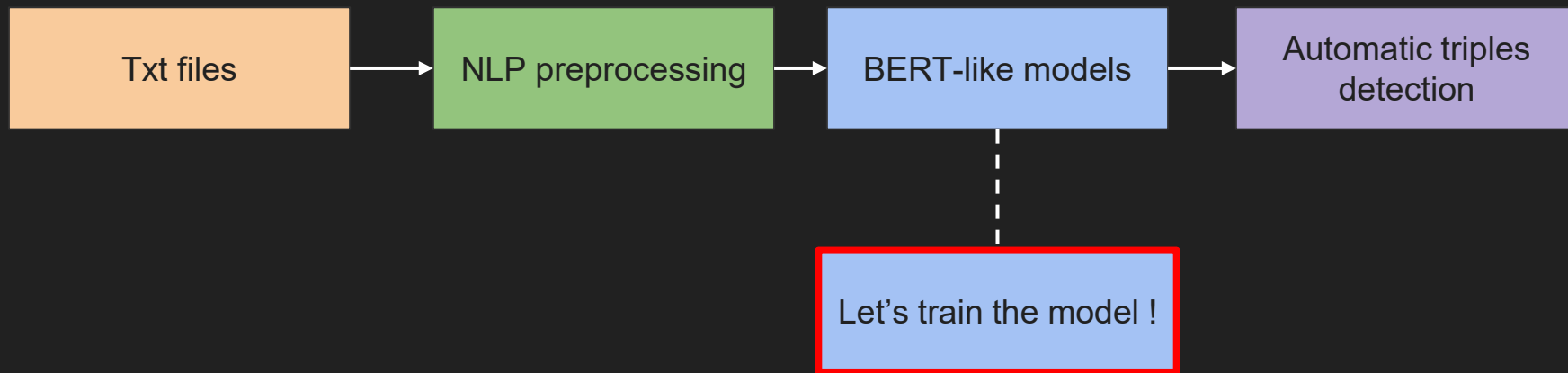
05

**Results &
Analysis**

06

Conclusion

Model training



⇒ Now that we have the dataset, let's train models and then inject them into the pipeline to evaluate them !

Model training

Use of pre trained models :

- *bert-base-uncased* (basic BERT, baseline)
- *roberta-base* (stronger pretraining (better objective than BERT))
- *distilbert-base-uncased* (smaller, faster to train)
- *albert-base-v2* (fewer parameters, good speed)
- *electra-base-discriminator* (efficient discriminator (replaced-token detection))

ML/DL methods (fine-tuning):

- Can use scheduler for optimizer parameters (learning rates, ...)
- loss is the loss used for the pre trained models
- Each epoch, computed loss and accuracy for the training and the validation datasets.

Model example: bert-base-uncased (baseline)

```
BertForSequenceClassification(
  (bert): BertModel(
    (embeddings): BertEmbeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (token_type_embeddings): Embedding(2, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (encoder): BertEncoder(
      (layer): ModuleList(
        (0-11): 12 x BertLayer(
          (attention): BertAttention(
            (self): BertSelfAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): BertSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
        )
      )
      (intermediate): BertIntermediate(
        (dense): Linear(in_features=768, out_features=3072, bias=True)
        (intermediate_act_fn): GELUActivation()
      )
      (output): BertOutput(
        (dense): Linear(in_features=3072, out_features=768, bias=True)
        (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
        (dropout): Dropout(p=0.1, inplace=False)
      )
    )
  )
  (pooler): BertPooler(
    (dense): Linear(in_features=768, out_features=768, bias=True)
    (activation): Tanh()
  )
  (dropout): Dropout(p=0.1, inplace=False)
  (classifier): Linear(in_features=768, out_features=8, bias=True)
)
```

Model example: roberta-base (stronger pretraining)

```

RobertaForSequenceClassification(
  (roberta): RobertaModel(
    (embeddings): RobertaEmbeddings(
      (word_embeddings): Embedding(50265, 768, padding_idx=1)
      (token_type_embeddings): Embedding(1, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
      (position_embeddings): Embedding(514, 768, padding_idx=1)
    )
    (encoder): RobertaEncoder(
      (layer): ModuleList(
        (0-11): 12 x RobertaLayer(
          (attention): RobertaAttention(
            (self): RobertaSelfAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): RobertaSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
          (intermediate): RobertaIntermediate(
            (dense): Linear(in_features=768, out_features=3072, bias=True)
            (intermediate_act_fn): GELUActivation()
          )
          (output): RobertaOutput(
            (dense): Linear(in_features=3072, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
    )
  )
  (classifier): RobertaClassificationHead(
    (dense): Linear(in_features=768, out_features=768, bias=True)
    (dropout): Dropout(p=0.1, inplace=False)
    (out_proj): Linear(in_features=768, out_features=8, bias=True)
  )
)

```

Model example: distilbert-base-uncased (smaller)

```

DistilBertForSequenceClassification(
  (distilbert): DistilBertModel(
    (embeddings): Embeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (transformer): Transformer(
      (layer): ModuleList(
        (0-5): 6 x TransformerBlock(
          (attention): DistilBertSelfAttention(
            (q_lin): Linear(in_features=768, out_features=768, bias=True)
            (k_lin): Linear(in_features=768, out_features=768, bias=True)
            (v_lin): Linear(in_features=768, out_features=768, bias=True)
            (out_lin): Linear(in_features=768, out_features=768, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (sa_layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
          (ffn): FFN(
            (dropout): Dropout(p=0.1, inplace=False)
            (lin1): Linear(in_features=768, out_features=3072, bias=True)
            (lin2): Linear(in_features=3072, out_features=768, bias=True)
            (activation): GELUActivation()
          )
        )
        (output_layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      )
    )
  )
  (pre_classifier): Linear(in_features=768, out_features=768, bias=True)
  (classifier): Linear(in_features=768, out_features=8, bias=True)
  (dropout): Dropout(p=0.2, inplace=False)
)

```

Model example: albert-base (fewer parameters)

```

AlbertForSequenceClassification(
  (albert): AlbertModel(
    (embeddings): AlbertEmbeddings(
      (word_embeddings): Embedding(30000, 128, padding_idx=0)
      (position_embeddings): Embedding(512, 128)
      (token_type_embeddings): Embedding(2, 128)
      (LayerNorm): LayerNorm((128,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0, inplace=False)
    )
    (encoder): AlbertTransformer(
      (embedding_hidden_mapping_in): Linear(in_features=128, out_features=768, bias=True)
      (albert_layer_groups): ModuleList(
        (0): AlbertLayerGroup(
          (albert_layers): ModuleList(
            (0): AlbertLayer(
              (full_layer_layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
              (attention): AlbertAttention(
                (attention_dropout): Dropout(p=0, inplace=False)
                (output_dropout): Dropout(p=0, inplace=False)
                (query): Linear(in_features=768, out_features=768, bias=True)
                (key): Linear(in_features=768, out_features=768, bias=True)
                (value): Linear(in_features=768, out_features=768, bias=True)
                (dense): Linear(in_features=768, out_features=768, bias=True)
                (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
              )
              (ffn): Linear(in_features=768, out_features=3072, bias=True)
              (ffn_output): Linear(in_features=3072, out_features=768, bias=True)
              (activation): NewGELUActivation()
              (dropout): Dropout(p=0, inplace=False)
            )
          )
        )
      )
      (pooler): Linear(in_features=768, out_features=768, bias=True)
      (pooler_activation): Tanh()
    )
    (dropout): Dropout(p=0.1, inplace=False)
    (classifier): Linear(in_features=768, out_features=8, bias=True)
  )
)

```

Model example: electra-base-discriminator (efficient discriminator)

```

ElectraForSequenceClassification(
  (electra): ElectraModel(
    (embeddings): ElectraEmbeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (token_type_embeddings): Embedding(2, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (encoder): ElectraEncoder(
      (layer): ModuleList(
        (0-11): 12 x ElectraLayer(
          (attention): ElectraAttention(
            (self): ElectraSelfAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): ElectraSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
          (intermediate): ElectraIntermediate(
            (dense): Linear(in_features=768, out_features=3072, bias=True)
            (intermediate_act_fn): GELUActivation()
          )
          (output): ElectraOutput(
            (dense): Linear(in_features=3072, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
    )
  )
  (classifier): ElectraClassificationHead(
    (dense): Linear(in_features=768, out_features=768, bias=True)
    (activation): GELUActivation()
    (dropout): Dropout(p=0.1, inplace=False)
    (out_proj): Linear(in_features=768, out_features=8, bias=True)
  )
)

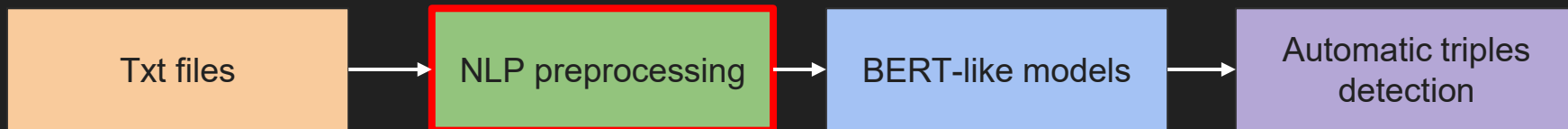
```

Recap of the pipeline



- 10 stories of 30 lines each from the previous work (benchmark unsupervised)
- Annotations in JSON, we have outputs triples for the evaluation at the end

Recap of the pipeline



- Entity recognition (dictionary based on the OWL = ontology)
- Named entities E1 or E2 + domains and range for multiple E1 and E2
- Sentence includes now the mention of E1 and E2 for the BERT-models

Bulbasaur is a Grass and Poison type, at least in his notes.

Bulbasaur **POKEMON_SPECIES** is a **Grass TYPE** and **Poison TYPE** type, at least in his notes.

ACTUAL TRIPLES:

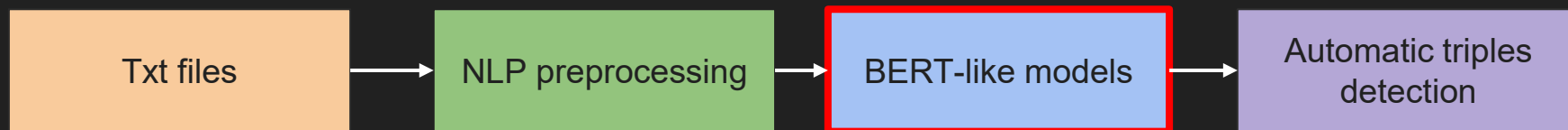
```
('http://example.org/pokemon#species_0001', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Grass')  
( 'http://example.org/pokemon#species_0001', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Poison')
```

PREDICTED TRIPLES:

```
('http://example.org/pokemon#species_0001', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Grass')  
( 'http://example.org/pokemon#species_0001', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Poison')
```

OK

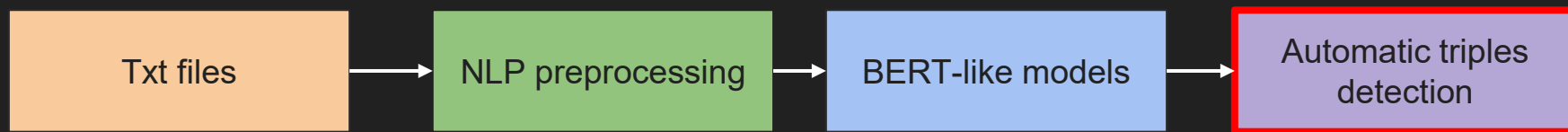
Recap of the pipeline



- Input :
Text annotated with entity E1/E2, vectorization of the whole sentence
- Output :
Classification problem so it will give us a class output

Then, we know both entities, the orders between them (E1 and E2) so we can create the triples E1, relation, E2 !

Recap of the pipeline



- BERT-like model gave us a class as the output

Then, we know both entities, the orders between them (E1 and E2) so we can create the triples E1, relation, E2 ! (if output class is not the None class)

Table of content

01

**Introduction &
Pipeline overview**

02

**Dataset
creation**

03

Model training

04

**Evaluation
metrics**

05

**Results &
Analysis**

06

Conclusion

Evaluation metrics

In this case, relation is a classification problem at the triple level.

We compare then the actual vs predicted triples from each sentence from each story

- **TP (True Positive)**: predicted triple exists in actual
- **FP (False Positive)**: predicted triple not in actual
- **FN (False Negative)**: actual triple not predicted
- **Precision** = $TP / (TP + FP)$

“How many predicted relations are correct?”

- **Recall** = $TP / (TP + FN)$

“How many actual relations did we recover?”

- **F1** = $2PR / (P + R) \rightarrow$ important

Balance between precision and recall

Table of content

01

**Introduction &
Pipeline overview**

02

**Dataset
creation**

03

Model training

04

**Evaluation
metrics**

05

**Results &
Analysis**

06

Conclusion

Results bert-base-uncased model (baseline)

Story	TP	FP	FN	Precision (%)	Recall (%)	F1 (%)
story_01	9	4	0	69.2 %	100 %	81.8 %
story_02	10	5	0	66.7 %	100 %	80 %
story_03	10	1	1	76.9 %	90.9 %	83.3 %
story_04	11	3	3	78.6 %	78.6 %	78.6 %
story_05	10	0	7	76.9 %	58.8 %	66.7 %
story_06	14	2	3	87.5 %	82.4 %	84.8 %
story_07	12	5	4	70.6 %	75 %	72.7 %
story_08	10	2	5	83.3 %	66.7 %	74.1 %
story_09	13	2	4	86.7 %	76.5 %	81.2%
story_10	18	4	6	81.8 %	75 %	78.3 %
TOTAL (mean)	-	-	-	78 %	78 %	78 % 25

Results roberta-base model (stronger pretraining)

Story	TP	FP	FN	Precision (%)	Recall (%)	F1 (%)
story_01	9	4	0	69.2 %	100 %	81.8 %
story_02	10	6	0	62.5 %	100 %	76.9 %
story_03	11	4	0	73.3 %	100 %	84.6 %
story_04	12	4	2	75 %	85.7 %	80 %
story_05	14	3	3	82.4 %	82.4 %	82.4 %
story_06	17	2	0	89.5 %	100 %	94.4 %
story_07	15	6	1	71.4 %	93.8 %	81.1 %
story_08	14	2	1	87.5 %	93.3 %	90.3 %
story_09	17	3	0	85.0 %	100 %	91.9 %
story_10	23	4	1	85.2 %	95.8 %	90.2 %
TOTAL (mean)	-	-	-	78.9 %	94.7 %	86.1 % 26

Results distilbert-base-uncased (smaller)

Story	TP	FP	FN	Precision (%)	Recall (%)	F1 (%)
story_01	9	4	0	69.2 %	100 %	81.8 %
story_02	10	3	0	76.9 %	100 %	87.0 %
story_03	10	3	1	76.9 %	90.9 %	83.3 %
story_04	10	3	4	76.9 %	71.4 %	74.1 %
story_05	11	3	6	78.6 %	64.7 %	71.0 %
story_06	14	2	3	87.5 %	82.4 %	84.8 %
story_07	12	7	4	63.2 %	75.0 %	68.6 %
story_08	11	2	4	84.6 %	73.3 %	78.6 %
story_09	14	2	3	87.5 %	82.4 %	84.8 %
story_10	19	5	5	79.2 %	79.2 %	79.2 %
TOTAL (mean)	-	-	-	77.9 %	80.0 %	78.9 % 27

Results abert-base (fewer parameters)

Story	TP	FP	FN	Precision (%)	Recall (%)	F1 (%)
story_01	8	3	1	72.7 %	88.9 %	80.0 %
story_02	10	3	0	76.9 %	100 %	87.0 %
story_03	10	3	4	76.9 %	90.9 %	83.3 %
story_04	10	3	4	76.9 %	71.4 %	74.1 %
story_05	10	3	7	76.9 %	58.8 %	66.7 %
story_06	12	2	5	85.7 %	70.6 %	77.4 %
story_07	14	5	2	73.7 %	87.5 %	80.0 %
story_08	12	2	3	85.7 %	80.0 %	82.8 %
story_09	15	2	2	88.2 %	88.2 %	88.2 %
story_10	19	4	5	82.6 %	79.2 %	80.9 %
TOTAL (mean)	-	-	-	80.0 %	80.0 %	80.0 % 28

Results electra-base-discriminator (efficient discriminator)

Story	TP	FP	FN	Precision (%)	Recall (%)	F1 (%)
story_01	8	3	1	72.7 %	88.9 %	80.0 %
story_02	10	5	0	66.7 %	100 %	80.0 %
story_03	10	2	1	83.3 %	90.9 %	87.0 %
story_04	12	2	2	85.7 %	85.7 %	85.7 %
story_05	10	2	7	83.3 %	58.8 %	69.0 %
story_06	13	1	4	92.9 %	76.5 %	83.9 %
story_07	12	3	4	80.0 %	75.0 %	77.4 %
story_08	10	1	5	90.9 %	66.7 %	76.9 %
story_09	14	3	3	82.4 %	82.4 %	82.4 %
story_10	19	6	5	76.0 %	79.2 %	77.6 %
TOTAL (mean)	-	-	-	80.8 %	78.7 %	79.7 %
						29

Results

- 4 models (DistilBERT, ALBERT, ELECTRA, RoBERTa) reach 80% in precision, recall and f1-score
- RoBERTa-base: 95% recall and 86% F1, meaning it recovers more true relations.
- RoBERTa got more pretraining (dynamic masking) → better generalization
- It handles more lexical variation, so fewer false negatives

Table of content

01

**Creation of the
Dataset**

02

SpaCy model

03

**BERT-like
model**

04

Metrics

05

Results

06

Conclusion

Limitations

- Split: 250 - 50 so learn more in the explicit patterns (“can learn”, “evolves to”).
- Exact ontology label matching, no variant (“Pikachu” exactly (trim, whitespace, lower, ..., but not enough).
- Evaluation of the relation classification here, not on the entity detection ! So errors propagate.

Improvements

- Mix multiple entities per sentence to create ambiguity, implicit relation, false negatives (entities present but no relation)
- Add a better NER model (no propagation of false entity detection)
 - a. Supervised training of a AutoModelForTokenClassification (same BERTs but different task)
 - b. Format: B-X = beginning of an entity of type X | I-X = inside an entity of type X | O = not an entity

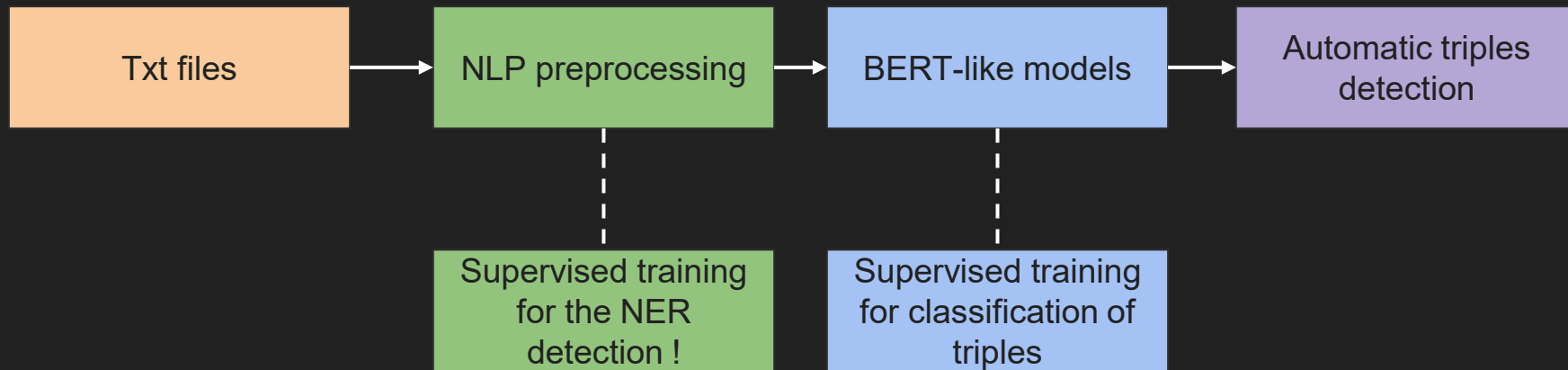
<u>Tokens:</u>	Pikachu	can	learn	Thunder
	Shock			
<u>Labels:</u>	B-POKEMON	O	O	B-MOVE

Improvements

- Mix multiple entities per sentence to create ambiguity, implicit relation, false negatives (entities present but no relation)
- Add a better NER model (no propagation of false entity detection)
 - a. Supervised training of a AutoModelForTokenClassification (same BERTs but different task)
 - b. Format: B-X = beginning of an entity of type X | I-X = inside an entity of type X | O = not an entity

<u>Tokens:</u>	Pikachu	can	learn	Thunder
	Shock			
<u>Labels:</u>	B-POKEMON	O	O	B-MOVE

Improvements



Expects better precision score but lower recall so overall better f1.

-> 80% for real text

Thank you for your attention !

Questions ?

Sources

- [Investigation of improving the pre-training and fine-tuning of BERT model for biomedical relation extraction | BMC Bioinformatics | Springer Nature Link](#)
- [Web Interface of NER and RE with BERT for Biomedical Text Mining](#)
- [Biomedical Text NER Tagging Tool with Web Interface for Generating BERT-Based Fine-Tuning Dataset](#)
- [Token classification](#)
- [general approach for improving deep learning-based medical relation extraction using a pre-trained model and fine-tuning | Database | Oxford Academic](#)